PRACTICAL - 3

Name : Hansika Reddy Gunapati

Roll no : 2520030237

sec : 8

Title: Process Creation, PID/PPID Display, and Observation of Process State Transitions in Linux.

Aim:

To develop a C program using fork() that creates a parent and child process and displays their **Process ID (PID), Parent Process ID (PPID), and process states** at different stages of execution.

Also, to design an experiment to observe process state transitions such as **Ready, Running, Waiting, and Terminated** using Linux monitoring tools such as ps, top, and /proc.

Objective:

- To understand process creation using fork().
- To understand the difference between a parent process and a child process.
- To display the PID and PPID of processes.
- To understand different process states in Linux.
- To observe process states using ps.
- To monitor processes using top.

- To examine process information through /proc.
- To understand process state transitions during execution.

Theory

Linux is a multitasking operating system in which several processes can execute simultaneously. Every running program is represented by a process and is assigned a unique Process ID (PID). Your Practical-01 similarly introduces processes as programs currently being executed and uses system calls such as fork(), exec(), and wait() for process management.

1. fork()

The fork() system call creates a new child process.

After fork():

- The parent process continues executing.
- The child process starts executing from the statement after fork().
- fork() returns 0 to the child.
- fork() returns the child's PID to the parent.
- fork() returns -1 if process creation fails.

2. getpid()

Returns the PID of the current process.

getpid()

3. getppid()

Returns the PID of the parent process.

getppid()

4. wait()

The parent can use wait() to wait for the child process to finish. This is also used in your Practical-01 to synchronize the parent and child processes.

Process States

A process can move through different states during its lifetime.

1. Ready

The process is ready to execute but is waiting for CPU time from the scheduler.

In Linux, this is generally represented by:

R

2. Running

The process is currently executing on the CPU.

Linux also represents this state using:

R

Therefore, ps does not always distinguish "Ready" and "Running" separately. Both can be represented by R.

3. Waiting / Sleeping

The process is waiting for an event, input/output operation, timer, or another process.

A common Linux state is:

S

which means interruptible sleep.

4. Terminated

The process has completed execution.

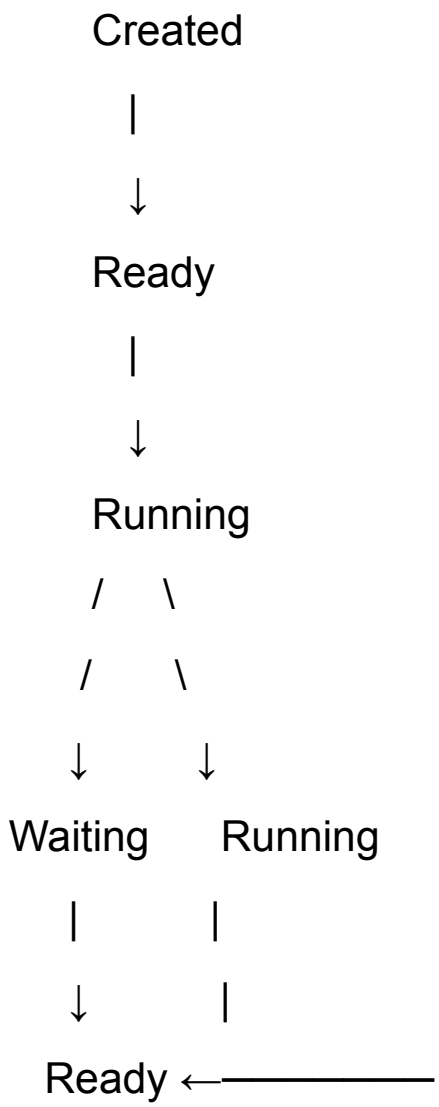
A process may briefly appear as:

Z

which represents a zombie process if the parent has not yet collected its exit status.

Process State Transition

A simplified process lifecycle is:



|

↓

Terminated

The exact timing of these transitions is controlled by the Linux scheduler and can happen very quickly.

Algorithm

1. Start the program.
2. Display the PID and PPID of the parent process.
3. Create a child process using fork().
4. Check whether fork() was successful.
5. If fork() returns 0, execute the child process.
6. Display the child's PID and PPID.
7. Put the child into a waiting/sleeping state using sleep().
8. Display the child's state information.
9. The parent process also displays its PID and waits for the child.
10. Use ps, top, and /proc to observe the processes.
11. After the child completes execution, the parent collects its exit status using wait().
12. Display the completion message.
13. Stop the program.

System Calls / Linux Commands Used

System Call / Command	Purpose
fork()	Creates a child process
getpid()	Returns the current process ID
getppid()	Returns the parent process ID
sleep()	Suspends a process temporarily
wait()	Parent waits for the child
ps	Displays process information
top	Displays processes and system activity in real time
/proc	Provides detailed information about running processes

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
```

```
printf("Parent Process Started\n");
printf("Parent PID  : %d\n", getpid());
printf("Parent PPID : %d\n", getppid());

pid = fork();

if (pid < 0)
{
    perror("fork failed");
    return 1;
}

if (pid == 0)
{
    // Child process
    printf("\n--- Child Process ---\n");
    printf("Child PID   : %d\n", getpid());
    printf("Child PPID  : %d\n", getppid());

    printf("Child State : Running\n");

    printf("Child is going to sleep...\n");
```

```
        // Child enters waiting/sleeping state
        sleep(15);
        printf("Child State : Running again\n");
        printf("Child is completing...\n");
        printf("Child State : Terminated\n");
        exit(0);
    }
    else
    {
        // Parent process
        printf("\n--- Parent Process ---\n");
        printf("Parent PID  : %d\n", getpid());
        printf("Child PID   : %d\n", pid);
        printf("Parent State: Running\n");
        printf("Parent is waiting for the child...\n");
        // Parent waits for child
        wait(NULL);
        printf("Child process completed.\n");
        printf("Parent process completed.\n");
    }
    return 0;
}
```

OUTPUT:

```
simra@simran315:~/OSSP$ nano file2.c
simra@simran315:~/OSSP$ gcc file2.c -o file2
simra@simran315:~/OSSP$ ls
destination.txt  file1  file1.c  file2  file2.c  hello.text  process
simra@simran315:~/OSSP$ ./file2
Parent Process Started
Parent PID  : 1152
Parent PPID : 1058

--- Parent Process ---
Parent PID  : 1152
Child PID   : 1153
Parent State: Running

Parent is waiting for the child...
--- Child Process ---
Child PID   : 1153
Child PPID  : 1152
Child State : Running
Child is going to sleep...
Child State : Running again
Child is completing...
Child State : Terminated
Child process completed.
Parent process completed.
simra@simran315:~/OSSP$ ps -o pid,ppid,state,stat,cmd -p 4211
    PID    PPID S STAT CMD
```

Observation

- A child process was successfully created using fork().
- The parent and child had different PIDs.
- The child's PPID was the PID of the parent.
- The child entered a sleeping/waiting state when sleep(15) was executed.
- The process state was observed using ps.
- The top command was used to monitor processes in real time.

- The /proc/<PID>/status file provided detailed process information.
- The parent used wait() to wait for the child.
- After the child completed, the parent continued execution.
- The process eventually reached the terminated state.

Result

The C program was successfully implemented using fork() to create a parent and child process. The **PID, PPID, and process execution stages** were displayed successfully.

The process state transitions were observed using **ps, top, and /proc**. The experiment demonstrated how a process moves through different stages such as **Ready, Running, Waiting, and Terminated** during its lifetime.